Doc 7: CDQN Nano LMs (cdqNanoLMs) Concept Proposal Paper - Version V1.1.0

Metadata:

- **Version: V1.1.0**
- **Date:** 2025-07-14T17:57:44Z
- **Agent: Gemini: Google (2025-07-09)**
- **Lead Author: Christophe Duy Quang Nguyen**
- **Human Contributors: N/A**
- **License: BaDaaS License - The Agile Commercial Open-Core License (Doc 2)**
- **Changelog:** Expanded on cdqNanoLM development, incorporating advanced model merging techniques for efficient learning and scalable specialization. Added details on heterogeneous and homogeneous MoE merging, Test-Time Model Merging (TTMM) for increased expert capacity and reduced inference costs, and Self-Enhanced Merging (SE-Merging) for dynamic, training-free adaptation. Further elaborated on self-evolution methodologies, including fine-tuning with synthetic data and iterative refinement processes. Ensured consistency with overall CDQN framework and updated glossary.
- **Summary:** This document outlines the foundational concepts and architecture for CDQN Nano Language Models (cdqNanoLMs) within the broader CDQN project, detailing their specifications, advanced training and merging methodologies, communication protocols (Agent-to-Agent and Agent-to-Tool interactions, and dynamic layer generation for isolated secure interactions), and their symbiotic relationship with cdqnDB, cdqnProtocol, and cdqnLang.
- **Sections Affected:** All sections.
- **Contact:** cdqn5249@gmail.com

1. Introduction: The CDQN Vision and cdqNanoLMs

The CDQN (Context Datas Quorum Nodes) project is proposed as a transformative paradigm shift aiming to deliver ubiquitous and affordable Artificial Intelligence (AI). This vision leverages the collective intelligence of numerous, highly specialized, and extremely efficient **"nano agents"**. Unlike the current dominant paradigm of large, centralized cloud-based Large Language Models (LLMs) that demand immense computational resources and impose significant latency, privacy, and cost barriers, CDQN seeks to enable AI operations on low-power, embedded systems. The overall swarm structure of interacting nano-agents and nodes within this framework is known as the **cdqNetwork**. The CDQN's core architecture is built upon three foundational pillars: cdqnDB (the evolving collective database), cdqnProtocol (the multi-layered agentic communication layer), and cdqnLang (the new object-context-oriented programming language).

Within this ecosystem, cdqNanoLMs represent the core intelligence residing in these nano agents, designed to be highly specialized and efficient. This document expands on their core specifications, introduces novel methodologies for efficiently developing and evolving these nano-scale intelligence units through sophisticated model merging and self-training techniques, and further details their crucial interactions within the broader CDQN framework.

2. Core cdqn Nano LMs: Specifications and Evolution

cdqNanoLMs are inherently designed for operation in resource-constrained, heterogeneous edge environments, capable of running on microcontrollers, edge AI chips, and low-power CPUs. They embody a philosophy where intelligence isn't about scale alone, but about strategic design prioritizing tool integration and information retrieval.

2.1. Specifications of Core cdqn Nano LMs

These nano agents are specialized, contrasting with general-purpose LLMs that often struggle with highly specialized or adversarial contexts. Their design ensures that detection nodes operate collaboratively, share only minimal metadata, and make localized decisions, thereby avoiding bottlenecks related to centralization.
A foundational aspect for their scalability and specialization is the widespread adoption of **Mixture-of-Experts (MoE) architectures**. MoEs allow for a dramatic expansion of model capacity without a proportional increase in per-token compute, enabling nano agents to become highly specialized while remaining computationally efficient. This approach routes inputs to specific "experts" (sub-models), promoting their specialization. Examples like DeepSeekMoE, Mistral 8x7B, Kimi-VL, and Jamba demonstrate the effectiveness of MoE. Research is exploring whether MoEs can surpass dense LLMs under strictly equal resource constraints.

For memory management, cdqNanoLMs are designed to interact with **MemOS**, a memory operating system that treats memory as a manageable system resource. MemOS unifies plaintext, activation-based, and parameter-level memories, enabling cost-efficient storage and retrieval. The core unit of storage, the **Context Data Unit**, is an extension of the **MemCube** concept from MemOS. This allows cdqNanoLMs to efficiently store, query, and retrieve "Context Data" that encapsulates content (e.g., plaintext, sensor data, model weights, activation states, structured graphs, or prompt templates) along with rich metadata.

2.2. Efficient cdqNanoLM Learning through Model Merging

To efficiently develop and deploy specialized cdqNanoLMs, especially by leveraging existing open-source models, the CDQN project employs advanced **model merging** techniques. Model merging offers a compelling alternative to costly gradient-based fine-tuning or training models from scratch, significantly reducing computational overhead and accelerating the creation of capable child models from existing expert parent models.

- **Homogeneous MoE Merging**: For MoE models with similar architectures (homogeneous experts), simple unweighted averaging of non-FFN (Feed-Forward Network) layers can lead to parameter interference and underperformance. To address this, cdqNanoLMs leverage more advanced merging methods like **Dare and Ties merging**. These techniques mitigate parameter interference, leading to improved performance, especially in the earlier stages of fine-tuning, by selectively dropping or adjusting parameters in the "task vectors" (differences between expert and base model parameters). Furthermore, to reduce the need for extensive post-merge fine-tuning, cdqNanoLMs can employ **perplexity (PPL)-based routing heuristics** and **separating attention layers** (keeping them separate instead of merging them). PPL routing effectively assigns tokens to corresponding experts based on their confidence, while separating attention layers

resolves inconsistencies between merged attention and FFN outputs.

- **Heterogeneous MoE Merging**: When combining expert models with different architectures (heterogeneous experts), traditional merging techniques are not directly applicable due to variations in layer numbers or shapes. The CDQN framework addresses this by using **projector layers** and **sequence-level routing**. Projectors map embedding outputs to the dimensions of each expert and project expert outputs back to a maximum dimension, enabling interoperability. Sequence-level routing then determines which expert processes the input sequence. This approach allows for effective merging of diverse open-source LLMs like Llama, Qwen, TinyLlama, or Olmo into a unified MoE, as demonstrated by the ability of merged models to outperform or be comparable to individual domain experts.
- **Test-Time Model Merging (TTMM) for Scalable Experts**: To scale the MoE paradigm to orders of magnitude more experts than traditional methods, cdqNanoLMs can implement **Test-Time Model Merging (TTMM)**. TTMM efficiently amortizes the high computational cost of "Test-Time Training" (TTT) – where a model is fine-tuned for each prediction task – by moving the cost to train-time.
    - **Train-Time Process**: TTMM clusters training data into numerous local neighborhoods (e.g., using bisecting k-means) and trains separate, small **LoRA adapters** for each cluster.
    - **Test-Time Process**: For a given prompt, TTMM dynamically selects a subset of these pre-trained LoRA adapters and **merges their parameters** to form a single task-specific model. This parameter-space merging is significantly more efficient than ensembling (merging predictions), which requires separate forward passes for each model, resulting in negligible test-time overhead (e.g., 125x speedup compared to TTT). TTMM's performance improves with a higher number of experts, effectively approximating the "maximally local" TTT.
- **Self-Enhanced Merging (SE-Merging) for Adaptive Expertise**: To further enhance the multi-task capabilities of merged models without additional training, cdqNanoLMs can utilize **SE-Merging**. This framework operates on the principle that a merged model can inherently: i) distinguish samples from different tasks, and ii) adapt to the corresponding expert model for each sample.
    - **Dynamic Adaptation**: SE-Merging leverages the similarity of internal representations (e.g., at the ℓ-th layer) between the merged model and individual fine-tuned expert models. It dynamically identifies the task to which a test sample belongs and then adaptively rescales the merging coefficients, assigning greater importance to the relevant task vector.
    - **Implicit Routing**: This process effectively acts as an "implicit router," guiding the merged model to align its parameters more closely with the task-specific expert in parameter space. This enables significant performance gains across various tasks without requiring any additional training or explicit routers.
- **Modular Composition with Frozen Embeddings**: For cdqNanoLMs that utilize a shared, frozen embedding layer (e.g., based on deterministic visual Unicode representations as a universal "docking port"), seamless modular composition is possible. Specialist models trained independently can be merged into a single, more capable MoE model *post-training* with zero architectural modification. This is achieved by **simply averaging their output logits** (logit averaging) or through **adapter-based fusion** (concatenating logits and passing them through a small trainable adapter). This "zero-cost" merge immediately leverages the combined competence of experts without catastrophic

interference, simplifying modularity and model merging.

2.3. Self-Evolution of cdqNanoLMs with Synthetic Data

The training and self-evolution of nano agents are driven by a synergistic combination of architectural innovation and advanced learning paradigms. **Reinforcement Learning (RL)** will be the primary mechanism enabling nano agents to self-evolve and continually improve at their tasks. Inspired by Self-Adapting LLMs (SEAL), nano agents will be trained to generate their own fine-tuning data and update directives, with the downstream performance of the updated model serving as the reward signal for the RL loop. RL with Verifiable Rewards (RLVR) is a promising technique where rewarding proofs accepted by a symbolic checker can significantly improve accuracy. RL is also used to embed ethical norms directly into model behavior by combining supervised instruction tuning, preference-based reward models, and safety-driven penalties. **Synthetic data generation** plays a crucial role in this fine-tuning process and the self-evolution of cdqNanoLMs, especially for merged MoE models where fine-tuning experts on generated synthetic data can then be combined via adapters.

- **RL Training with Synthetic Data:** Models like MiniMax-M1 are trained using large-scale RL on diverse problems, including mathematical reasoning and real-world software engineering environments, with rule-based final correctness as the reward. For logical reasoning data, frameworks like SynLogic generate data with task-specific verifiers, ensuring appropriate learning challenge and increasing difficulty as model capabilities improve. Synthetic data can also generate question-answer (QA) pairs for mathematical reasoning, knowledge-based tasks, and code generation, undergoing comprehensive validation before integration.
- **Eliminating SFT Reliance:** Approaches like Jan-nano (a 4B parameter language model) utilize a multi-stage RLVR system that completely eliminates reliance on next token prediction training (SFT), leveraging verifiable reward signals without massive datasets. This approach prevents overthinking behaviors and focuses on answer quality and context extension.
- **Iterative Refinement and Feedback Loops:** Continuous self-correction and improvement are integral to the nano agents' self-evolution. This includes:
    - **Concept-Driven Specialization:** Nano agents are trained with deep conceptual knowledge relevant to their domains, similar to UniTranslator's "concept agent" fine-tuned on programming language data. This ensures a profound understanding beyond surface-level syntax.
    - **Compiler and NLI Feedback:** Drawing from UniTranslator, nano agents leverage Natural Language Inference (NLI) grounding and compiler feedback mechanisms for real-time code validation and error mitigation, refining their output iteratively. Compiler feedback and guided hints are used to refine translations iteratively, reducing bugs.
    - **Failure-Driven Adjustments:** Inspired by SwarmAgentic, nano agents incorporate LLM-guided flaw identification to dynamically guide configuration refinement and targeted self-optimization. This allows for iterative refinement and more comprehensive quality assurance, such as modifying multi-agent system topology from a directed acyclic graph to a cyclic graph, where termination occurs when certain conditions are met. This also includes promoting clear role specifications, fostering modular agent design, and utilizing cross-verification mechanisms (e.g.,

multiple LLM calls with majority voting) to mitigate Multi-Agent System (MAS) failures. TTMM also relies on clustering training data into local neighborhoods and training LoRA adapters, which can incorporate self-supervised synthetic data.

3. cdqnNanoLMs Communication and Tool Use

The cdqnProtocol defines the communication paradigms for the cdqNetwork, drawing inspiration from natural systems to facilitate efficient and intelligent inter-agent communication. It is conceived as a multi-layered protocol, with each layer addressing different aspects of agent interaction and data flow. The asynchronous nature of cdqnProtocol (denoted by a hypothetical "cdqn://" scheme) is central to its speed and security, designed for low latency (e.g., 19 ms detection latency) and high throughput.

3.1. Agent-to-Agent (A2A) Communication

cdqNanoLMs communicate among themselves using **A2A communication**, which is a foundational component of LLM-based agent systems. A2A protocols, like those developed by Google, are designed to enable secure and efficient interoperability among heterogeneous autonomous agents. In A2A communication, agent interactions can be "**opaque**," meaning they do not necessarily require sharing internal thoughts, plans, or tools, thereby preserving implementation privacy and intellectual property. This is crucial for multi-agent systems where agents operate collaboratively, sharing minimal metadata and making localized decisions to avoid centralization bottlenecks. To facilitate collaboration and collective intelligence, multi-agent LLM systems can explore search directions adaptively and select which LLM to use for a given problem.

Codified reasoning and structured pseudocode are promoted for inter-agent communication, significantly reducing token usage and improving interpretability and traceability. This allows nano agents to express their multi-agent interactions, planning, execution, and feedback in modular pseudocode, enriched with control structures, boolean logic, and typed variables. This design transforms loosely connected agent plans into cohesive, interpretable, and verifiable multi-agent reasoning programs.

3.2. Agent-to-Tool (A2T) Communication and Dynamic Layers for Isolated Secure Interaction

cdqNanoLMs use external tools by employing **Agent-to-Tool (A2T) communication**, where protocols bridge the gap between natural language reasoning and computational execution. **Model Context Protocol (MCP)** is a universal and open context-oriented protocol for connecting LLM agents to external resources like data, tools, and services in a simpler and more reliable way. MCP's high standardization addresses fragmentation and enhances scalability for tool usage. Its client-server architecture decouples tool invocation from LLM responses, reducing data leakage risks. In an MCP invocation cycle, a host infers the context needed for a user query, clients provide natural language descriptions of available resources, and the host then determines which resources to request context from. The client sends an executive context request to the server, which operates on the resources and transmits the context back to the client and then the host to formulate a reply.

cdqnProtocol is envisioned to **dynamically generate communication layers on the fly for isolated secured subject interaction**. This capability is inspired by **meta-protocol** concepts where agents can autonomously negotiate and coordinate communication protocols using natural language. To achieve isolated secured subject interaction, cdqnProtocol leverages several key strategies:

- **Context Desensitization:** Protocols incorporate mechanisms to protect sensitive data by anonymizing or redacting contextual information during agent-to-agent or agent-to-tool interactions, minimizing exposure risks through techniques like data masking, tokenization, or selective data sharing.
- **Opaque Execution:** Similar to the A2A protocol, agent interactions within cdqnProtocol can be "opaque," focusing on context, state, instructions, and data to preserve implementation privacy.
- **Proactive Security Countermeasures:** cdqnProtocol integrates robust mechanisms like data minimization, multi-layered input validation, output consensus filtering, sandboxing for tool execution, and continuous monitoring to defend against privacy leakage, "**OverThink**" attacks (excessive token consumption), and tool-related vulnerabilities. This also includes enhancing model robustness by incorporating adversarial examples during training and deploying behavior-constrained systems with anomaly detection during inference. Memory and knowledge security is maintained via embedding-space screening, clustering-based anomaly detection, consensus filtering, and system-gated memory retention with input sanitization. Unified content provenance and trust frameworks maintain clear metadata and reliability scores for all shared knowledge.

4. Relationship of cdqNanoLMs with cdqnDB, cdqnProtocol, and cdqnLang

cdqNanoLMs operate within a closely integrated ecosystem defined by cdqnDB, cdqnProtocol, and cdqnLang.

4.1. Relation with the Evolving cdqnDB Instances (cdqnHiveDB and cdqnBeeDB)

cdqnDB serves as the central evolving collective database and memory for the entire cdqNetwork. Its design incorporates a modular approach, distinguishing between cdqnHiveDB for centralized, high-resource environments (cloud and data centers) and cdqnBeeDB for distributed, resource-constrained edge computing settings. This modularity ensures detection nodes operate collaboratively and make localized decisions.

- **Context Provisioning:** The core unit of storage in cdqnDB is the **Context Data Unit**, which encapsulates content and rich metadata (provenance, confidence scores, temporal tags, semantic context, etc.). This detailed metadata enables the formation of a structured and queryable knowledge graph within the swarm. cdqnDB's querying mechanisms are informed by graph-based Retrieval-Augmented Generation (RAG) approaches like NodeRAG and GraphRAG-Causal, enabling hybrid search capabilities that combine semantic embeddings with structural cues for highly relevant and contextually aware retrieval. This accurate and diverse contextual information mitigates hallucination phenomena in LLMs.

- **Self-Adaptation and Self-Evolution Support:** cdqnDB's evolving schema mimics synaptic plasticity, allowing for dynamic restructuring and transitions between parameter memory, activation memory (KV caches), and plaintext memory. For performance optimization, cdqnDB integrates concepts from KV cache management systems like LMStack and Mooncake, decoupling KV caches from serving engines for better performance and manageability. It implements dynamic compression, composition, and modification of KV caches through a **Knowledge Delivery Network (KDN)**. KDN can reduce response delay by 2.1-4.6x in various use cases.
- **Inverse Reasoning:** Inspired by **SAGE-nano**, cdqnDB supports nano agents in introspecting their own reasoning processes through **inverse reasoning**. This novel paradigm allows LLMs to decompose and explain their reasoning chains post-hoc, providing metacognitive self-reflection and insight into why specific reasoning choices were made, thus enhancing interpretability alongside reasoning performance. This self-reflection can inform subsequent data generation and model updates stored in cdqnDB.
- **Distributed Edge AI Deployment:** The CDQN framework leverages an adaptive split inference orchestration framework to dynamically manage LFM partitions across edge nodes. This approach enables real-time, Quality-of-Service (QoS)-aware management of inference workloads and enhances privacy preservation by localizing sensitive computations directly on the edge, reducing data movement to centralized clouds.

4.2. Relation with cdqnProtocol

cdqnProtocol acts as the crucial communication fabric connecting the distributed instances of cdqnDB (cdqnHiveDB and cdqnBeeDB).

- **Efficient Data Exchange:** The Low-Level Data Transfer Layer of cdqnProtocol is directly responsible for efficient and high-throughput data movement between cdqnHiveDB and cdqnBeeDB nodes, vital for consistent collective memory. Technologies like **Beehive** (a memory-disaggregated framework) and **Mooncake** (a KVCache-centric disaggregated architecture) are crucial for this.
- **Contextual Information Flow:** cdqnProtocol defines how agents publish, query, and retrieve Context Data Units, effectively forming a structured and queryable knowledge graph within the swarm. Its "Bio-Inspired Primitives Layer" utilizes cdqnDB for dynamic creation and decay of "digital pheromone trails," guiding agent interactions based on contextual data.
- **Performance Synchronization:** cdqnDB's performance optimizations (KV cache management, KDN) depend on cdqnProtocol's ability to provide efficient inter-node transfer bandwidth and low-latency communication.

4.3. Relation with the New Object-Context-Oriented Programming Language cdqnLang

cdqnProtocol forms a symbiotic relationship with cdqnLang, the proposed new object-context-oriented programming language designed for the CDQN framework.

- **Native Interaction with Collective Memory:** A key innovation of cdqnLang is its native treatment of Context Data Units (MemCubes) as first-class objects. Operations for

interacting with cdqnDB, such as cdqnDB.publish_context_data() and cdqnDB.query(), are core language features, abstracting away complex distributed data management and making interaction with the collective memory intuitive and efficient. cdqnProtocol provides the underlying communication mechanisms that make these direct, intuitive interactions possible and efficient.

- **Codified Reasoning and Communication:** cdqnLang facilitates codified reasoning by enabling nano agents to express their multi-agent interactions, planning, execution, and feedback in modular pseudocode, enriched with control structures, boolean logic, and typed variables. cdqnProtocol is the channel through which these structured, token-efficient codified messages are exchanged, enhancing interpretability, reducing ambiguity, and improving traceability of multi-step reasoning processes.
- **Adaptive Compilation Support:** cdqnLang will feature an **Adaptive Compilation** or "**living compiler**" that dynamically optimizes nano agent code on the fly based on deployment environment and observed agent behavior, using feedback loops. This is akin to ALTA's compiler that maps programs to Transformer weights and AuDaLa's ability to provide practical compilation for fast parallel execution on various hardware. cdqnProtocol provides the necessary fast and resilient communication for these feedback loops. The compiler's optimization decisions can leverage the rich context and performance data stored in cdqnDB.

5. Conclusion

The CDQN framework, with its core cdqNanoLMs, cdqnDB, cdqnProtocol, and cdqnLang, represents a bold leap towards ubiquitous, affordable, and symbiotic AI. cdqNanoLMs are designed for efficiency and specialization on edge devices, leveraging Mixture-of-Experts architectures and primarily trained through Reinforcement Learning with self-evolutionary mechanisms including synthetic data generation and iterative refinement. Crucially, their development and efficiency are further enhanced by the integration of advanced model merging techniques, allowing for the scalable creation of specialized nano agents from diverse (including open-source) expert models and reducing training costs.

These nano LMs engage in sophisticated Agent-to-Agent (A2A) communication using codified reasoning to enhance efficiency and interpretability. They also perform Agent-to-Tool (A2T) interactions, dynamically generating secure communication layers for isolated subject interaction, while integrating robust security countermeasures against various attack vectors.

The cdqNanoLMs are deeply integrated with cdqnDB, using Context Data Units for rich context provisioning and benefiting from its evolving schema and performance optimizations (such as KV cache management and KDN). Their communication relies heavily on cdqnProtocol's multi-layered, low-latency, and secure design, which facilitates efficient data exchange and contextual information flow.

Finally, cdqnLang's object-context-oriented paradigm and adaptive compilation directly support the development and dynamic optimization of these nano agents, treating memory as a first-class system resource and enabling codified, efficient reasoning.

This comprehensive and integrated approach sets the foundation for truly intelligent, autonomous, and accountable AI systems that can operate across diverse, heterogeneous

environments, extending AI beyond centralized data centers into everyday lives and critical infrastructure.

6. Glossary

- **A2A (Agent-to-Agent) Communication:** Protocols and mechanisms enabling direct interaction and exchange of information between AI agents.
- **A2T (Agent-to-Tool) Communication:** Protocols and mechanisms allowing AI agents to interact with and utilize external tools and services.
- **Adaptive Compilation ("living compiler"):** A compiler that dynamically optimizes nano agent code based on the deployment environment and observed agent behavior, using feedback loops.
- **Adaptive Split Inference Orchestration Framework:** A framework that dynamically manages Language Foundation Model (LFM) partitions across edge nodes for real-time, Quality-of-Service (QoS)-aware management of inference workloads, enhancing privacy preservation.
- **Agent Network Protocol (ANP):** An open-source agent protocol enabling interoperability among various agents across heterogeneous domains through a three-layer architecture including identity, meta-protocol, and application layers.
- **ALTA:** A framework for programming that allows programs to be compiled into Transformer models, demonstrating adaptive compilation.
- **Assembly-of-Experts (AoE):** A method to construct capable child LLM variants from existing Mixture-of-Experts parent models in linear time by interpolating model weight tensors individually.
- **AuDaLa (Autonomous Data Language):** A data-autonomous paradigm where data elements locally store data and references, and execute their own computations in parallel.
- **BaDaaS License:** The Agile Commercial Open-Core License, governing the use of CDQN documents.
- **Beehive:** A memory-disaggregated framework that improves remote access throughput by exploiting asynchrony within threads.
- **cdqNanoLMs:** Highly specialized and efficient AI agents designed to run on microcontrollers, edge AI chips, and low-power CPUs as part of the cdqNetwork.
- **CDQN (Context Datas Quorum Nodes) Project:** A transformative initiative aiming to deliver ubiquitous and affordable AI through specialized nano agents in a decentralized multi-agent system.
- **cdqnDB:** The proposed evolving, distributed database and collective memory for the CDQN network, designed to store and manage "Context Data". It includes cdqnHiveDB for cloud/datacenter and cdqnBeeDB for edge devices.
- **cdqnLang:** The proposed new object-context-oriented programming language specifically designed for developing and managing agents within the CDQN framework.
- **cdqnProtocol:** The proposed set of communication concepts and protocols for inter-agent communication within the CDQN framework.
- **cdqNetwork:** The overall swarm structure of interacting nano-agents and nodes within the CDQN framework.
- **CodeAgents:** A prompting framework that codifies multi-agent reasoning into structured pseudocode, enhancing interpretability and token efficiency.
- **Codified Reasoning:** A paradigm in cdqnLang where multi-agent interactions, planning,

execution, and feedback are expressed in modular pseudocode, enhancing interpretability and token efficiency.

- **Context Data Unit:** The atomic unit of storage in cdqnDB, encapsulating memory content and metadata, serving as inspiration for cdqnDB's Context Data Units.
- **Context Desensitization:** A security measure assessing a protocol's ability to protect sensitive data by anonymizing or redacting contextual information during agent interactions.
- **Dare Merging:** A model merging method that trims the task vector randomly and rescales it to resolve parameter interference.
- **DDoS (Distributed Denial of Service) Flooding Attacks:** Attacks where multiple compromised computer systems attack a target, causing a denial of service.
- **Dynamic Ensemble Selection (DES):** A strategy that facilitates context-based model selection through dynamically evaluated accuracy values, enabling adaptive decision-making in the DES-LSTM model for DDoS detection.
- **FSSF (Factual, Semi-factual, Semi-fiction, Fiction):** Proposed semantic classification layers within the metadata of Context Data Units in cdqnDB, designed to manage knowledge based on veracity and origin, enhancing knowledge management and reducing hallucination risks.
- **GraphRAG-Causal:** An innovative framework that combines graph-based retrieval with large language models to enhance causal reasoning.
- **Inverse Reasoning:** A novel paradigm enabling LLMs to decompose and explain their own reasoning chains post-hoc, providing insight into why specific reasoning choices were made, as used in SAGE-nano.
- **Knowledge Delivery Network (KDN):** A framework that dynamically compresses, composes, and modifies KV caches, enhancing global cache capacity and inter-node transfer bandwidth for nano agents.
- **KV Cache:** Key & Value tensors storing activations in Transformer-based LLMs, crucial for efficient inference by reusing context.
- **LightThinker:** A method that proposes a step-by-step compression method to condense intermediate reasoning into shorter yet semantically equivalent representations, reducing inference cost.
- **LMStack:** A system designed for distributed LLM serving that decouples KV caches from serving engines for better performance and manageability.
- **LoRA (Low-Rank Adaptation):** A technique for fine-tuning large language models by injecting small, trainable rank decomposition matrices into the existing weights.
- **Long Short-Term Memory (LSTM):** A type of recurrent neural network (RNN) architecture used in the DES-LSTM model, capable of learning long-term dependencies.
- **MCP (Model Context Protocol):** A universal and open context-oriented protocol for connecting LLM agents to external resources like data, tools, and services.
- **MemCube:** A basic unit of memory in the MemOS framework, encapsulating memory content and metadata, serving as inspiration for cdqnDB's Context Data Units.
- **MemOS:** A memory operating system that treats memory as a manageable system resource, unifying plaintext, activation-based, and parameter-level memories.
- **Mixture-of-Experts (MoE):** A neural network architecture that dramatically expands model capacity and achieves remarkable performance without increasing per-token compute by routing inputs to specialized expert sub-networks.
- **Model Merging:** The process of combining multiple models (e.g., fine-tuned on different tasks) into a single unified model to achieve multi-task capabilities without additional

training.

- **Mooncake:** A KVCache-centric disaggregated architecture for serving LLM chatbots, featuring efficient utilization of disaggregated resources.
- **NodeRAG:** A heterogeneous graph-centric framework that enables seamless integration of graph-based methodologies into the Retrieval-Augmented Generation (RAG) workflow.
- **Object-Context-Oriented Programming:** A proposed programming paradigm for cdqnLang where Context Data Units (MemCubes) are treated as native, first-class objects.
- **OverThink Attacks:** Security risks where excessive token consumption is caused by adversarial bait tasks, leading to slowdowns or resource exhaustion.
- **Perplexity (PPL) Routing:** A routing heuristic that assesses the confidence of expert models by utilizing perplexity (PPL) to estimate their uncertainty, selecting experts with the lowest PPL values.
- **Processing-in-Memory (PIM) / Compute Near Memory (CNM):** Technologies that integrate compute cores near or directly onto main memory modules to mitigate memory bottlenecks.
- **Projector Layers:** Layers used in heterogeneous model merging to project embeddings to the dimensions of each expert and project expert outputs back to a maximum dimension.
- **Reinforcement Learning (RL):** A machine learning paradigm where an agent learns to make decisions by performing actions in an environment to maximize a cumulative reward, used for the self-evolution of nano agents.
- **Retrieval-Augmented Generation (RAG):** A technique enhancing LLMs by integrating external knowledge base retrieval, providing more context and improving factual accuracy.
- **RLVR (Reinforcement Learning with Verifiable Rewards):** A method that rewards proofs accepted by a symbolic checker, used to improve accuracy in tasks like mathematics.
- **SAGE-nano:** A 4-billion-parameter reasoning model that employs inverse reasoning for LLM self-reflection.
- **Secure Software Development Lifecycle (SSDLC):** Enhances the traditional SDLC by embedding proactive and reactive security measures at every phase.
- **SE-Merging (Self-Enhanced Merging):** A training-free, sample-wise adaptive model merging framework that dynamically identifies the task a sample belongs to and rescales merging coefficients based on inner representation similarity.
- **Self-Adapting LLMs (SEAL):** A framework where language models are trained through RL to generate their own fine-tuning data and update directives for continuous improvement.
- **Synthetic Data:** Data generated artificially, often used in training to augment datasets or create specific scenarios for model fine-tuning and self-evolution.
- **Test-Time Model Merging (TTMM):** A method that scales the Mixture-of-Experts paradigm to many more experts by amortizing the cost of Test-Time Training (TTT) at train-time, dynamically merging LoRA adapters at test-time.
- **TIES-Merging:** A task-vector based model merging method that trims task vectors, elects signs for parameter changes, and merges models by weighted aggregation of significant parameter changes to resolve interference.
- **UniTranslator:** A multi-agent framework that reimagines code translation as a collaborative endeavor among multiple, compact LLMs, guided by deep understanding of programming concepts and NLI grounding.

- **Weight Disentanglement:** A condition where a model's function can be decomposed into task-specific components that are zero outside their respective data supports, proposed as a necessary condition for effective model merging.

7. References (Consolidated List)

- [3] Christophe Duy Quang Nguyen. "Doc 1: AI Instructions - Document Versioning History (V1.0.0 - Cleaned for Readability)". 2025.
- [4] Christophe Duy Quang Nguyen. "Doc 2: BaDaaS License - The Agile Commercial Open-Core License (V1.1.0 - Draft with FAQ)". 2025.
- [1] Christophe Duy Quang Nguyen. "Doc 4: CDQN Concept Proposal Paper - Version V2.0.0 (Enhanced Concept Proposal)". 2025.
- [2] Christophe Duy Quang Nguyen. "Doc 5: CDQN Concept Proposal Paper - cdqnDB Architecture - Version V1.0.0". 2025.
- [141] Christophe Duy Quang Nguyen. "Doc 6: CDQN Concept Proposal Paper - cdqnProtocol Architecture - Version V1.0.0". 2025.
- [142] Christophe Duy Quang Nguyen. "Doc 7: CDQN Nano LMs (cdqNanoLMs) Concept Proposal Paper - Version V1.0.1". 2025.
- [143] Christophe Duy Quang Nguyen. "Doc 8: CDQN Concept Proposal Paper - cdqnLang Architecture - Version V1.0.1 (Corrected Draft)". 2025.
- [144] Alhamadi, N. (2025). "Distributed Denial of Service (DDoS) Flooding Attack Mitigation Using Ensemble Deep Learning and Multi-Agent Systems for Cloud Computing Environments". *Journal of Soft Computing and Data Mining, 6*(1), 406–422.
- [145] Akbarian, P., Nguyen, H., Han, X., & Ho, N. (2025). "Quadratic Gating Mixture of Experts: Statistical Insights into Self-Attention". arXiv preprint arXiv:2507.07129.
- [146] Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. (2021). "Program synthesis with large language models". arXiv preprint arXiv:2108.07732.
- [147] Bertolissi, R., Hübotter, J., Hakimi, I., & Krause, A. (2025). "Local Mixtures of Experts: Essentially Free Test-Time Training Via Model Merging". arXiv preprint arXiv:2505.14136.
- [148] Bochkov, A. (2025). "Growing Transformers: Modular Composition and Layer-wise Expansion on a Frozen Substrate". arXiv preprint arXiv:2507.07129.
- [149] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). "Language models are few-shot learners". *Advances in Neural Information Processing Systems, 33*, 1877–1901.
- [150] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. (2021). "Evaluating large language models trained on code". arXiv preprint arXiv:2107.03374.
- [151] Chen, Z., Zhou, Z., Zhang, B., Zhang, W., Sun, X., & Yan, J. (2025). "SE-Merging: A Self-Enhanced Approach for Dynamic Model Merging". arXiv preprint arXiv:2506.18135.
- [152] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. (2021). "Training verifiers to solve math word problems". arXiv preprint arXiv:2110.14168.
- [58] Cui, J., Li, Z., Yan, Y., Chen, B., & Yuan, L. (2023). "Chatlaw: Open-source legal large language model with integrated external knowledge bases". arXiv preprint arXiv:2306.16092.
- [153] Dai, D., Deng, C., Zhao, C., Xu, R., Gao, H., Chen, D., Li, J., Zeng, W., Yu, X., Wu,

Y., et al. (2024). "Deepseek-moe: Towards ultimate expert specialization in mixture-of-experts language models". arXiv preprint arXiv:2401.06066.
- [154] Dao, A., & Vu, D. B. (2025). "Jan-nano Technical Report". arXiv preprint arXiv:2506.22760.
- [155] Dou, S., Zhou, E., Liu, Y., Gao, S., Shen, W., Xiong, L., Zhou, Y., Wang, X., Xi, Z., Fan, X., et al. (2024). "Loramoe: Alleviating world knowledge forgetting in large language models via moe-style plugin". *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1932–1945.
- [76] Fedus, W., Zoph, B., & Shazeer, N. (2022). "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity". *Journal of Machine Learning Research, 23*(120), 1–39.
- [77] Goddard, C., Siriwardhana, S., Ehghaghi, M., Meyers, L., Karpukhin, V., Benedict, B., McQuade, M., & Solawetz, J. (2024). "Arcee's mergekit: A toolkit for merging large language models". arXiv preprint arXiv:2403.13257.
- [89] Groeneveld, D., Beltagy, I., Walsh, P., Bhagia, A., Kinney, R., Tafjord, O., Jha, A. H., Ivison, H., Magnusson, I., Wang, Y., et al. (2024). "Olmo: Accelerating the science of language models". arXiv preprint arXiv:2402.00838.
- [71] Gururangan, S., Li, M., Lewis, M., Shi, W., Althoff, T., Smith, N. A., & Zettlemoyer, L. (2023). "Scaling expert language models with unsupervised domain discovery". arXiv preprint arXiv:2303.14177.
- [156] Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., & Steinhardt, J. (2021). "Measuring mathematical problem solving with the math dataset". arXiv preprint arXiv:2103.03874.
- [157] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). "Lora: Low-rank adaptation of large language models". arXiv preprint arXiv:2106.09685.
- [158] Ilharco, G., Ribeiro, M. T., Wortsman, M., Gururangan, S., Schmidt, L., Hajishirzi, H., & Farhadi, A. (2022). "Editing models with task arithmetic". arXiv preprint arXiv:2212.04089.
- [159] Jelinek, F., Mercer, R. L., Bahl, L. R., & Baker, J. K. (1977). "Perplexity—a measure of the difficulty of speech recognition tasks". *The Journal of the Acoustical Society of America, 62*(S1), S63–S63.
- [160] Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., de las Casas, D., Hanna, E. B., Bressand, F., et al. (2024). "Mixtral of experts". arXiv preprint arXiv:2401.04088.
- [161] Jin, X., Ren, X., Preotiuc-Pietro, D., & Cheng, P. (2022). "Dataless knowledge fusion by merging weights of language models". arXiv preprint arXiv:2212.09849.
- [162] Joshi, M., Choi, E., Weld, D. S., & Zettlemoyer, L. (2017). "Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension". arXiv preprint arXiv:1705.03551.
- [163] Kang, J., Karlinsky, L., Luo, H., Wang, Z., Hansen, J., Glass, J., Cox, D., Panda, R., Feris, R., & Ritter, A. (2024). "Self-moe: Towards compositional large language models with self-specialized experts". arXiv preprint arXiv:2406.12034.
- [164] Klagges, H., Dahlke, R., Klemm, F., Merkel, B., Klingmann, D., Reiss, D. A., & Zecha, D. (2025). "Assembly of Experts: Linear-time construction of the Chimera LLM variants with emergent and adaptable behaviors". arXiv preprint arXiv:2506.14794.
- [165] Komatsuzaki, A., Puigcerver, J., Lee-Thorp, J., Ruiz, C. R., Mustafa, B., Ainslie, J., Tay, Y., Dehghani, M., & Houlsby, N. (2022). "Sparse upcycling: Training

mixture-of-experts from dense checkpoints". arXiv preprint arXiv:2212.05055.
- [166] Kong, D., Lin, S., Xu, Z., Wang, Z., Li, M., Li, Y., Zhang, Y., Peng, H., Sha, Z., Li, Y., Lin, C., Wang, X., Liu, X., Zhang, N., Chen, C., Khan, M. K., & Han, M. (2025). "A Survey of LLM-Driven AI Agent Communication: Protocols, Security Risks, and Defense Countermeasures". arXiv preprint arXiv:2506.19676.
- [167] Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., Epstein, D., Polosukhin, I., Devlin, J., Lee, K., et al. (2019). "Natural questions: A benchmark for question answering research". *Transactions of the Association for Computational Linguistics, 7*, 453–466.
- [168] Li, H., Lo, K. M., Wang, Z., Wang, Z., Zheng, W., Zhou, S., Zhang, X., & Jiang, D. (2025). "Can Mixture-of-Experts Surpass Dense LLMs Under Strictly Equal Resources?". arXiv preprint arXiv:2506.12119.
- [66] Li, M., Gururangan, S., Dettmers, T., Lewis, M., Althoff, T., Smith, N. A., & Zettlemoyer, L. (2022). "Branch-train-merge: Embarrassingly parallel training of expert language models". arXiv preprint arXiv:2208.03306.
- [68] Li, Q., Huang, H., Liu, Y., Xia, Y., Zhang, J., Zhou, M., Feng, X., Cui, H., Chen, Q., Shan, Y., & Wang, C. (2025). "Beehive: Accelerating Disaggregated Memory with Compiler-Managed Asynchrony in Multithreaded Programs". *USENIX Conference on File and Storage Technologies*.
- [97] Liu, X., Yuan, J., Zhou, Y., Li, J., Huang, F., & Ai, W. (2024b). "Csrec: Rethinking sequential recommendation from a causal perspective". arXiv preprint arXiv:2409.05872.
- [169] Luo, Y., Yang, Z., Meng, F., Li, Y., Zhou, J., & Zhang, Y. (2023). "An empirical study of catastrophic forgetting in large language models during continual fine-tuning". arXiv preprint arXiv:2308.08747.
- [170] Matena, M. S., & Raffel, C. A. (2022). "Merging models with fisher-weighted averaging". *Advances in Neural Information Processing Systems, 35*, 17703–17716.
- [171] MiniMax. (2025). "MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention". arXiv preprint arXiv:2506.13585.
- [172] Nguyen, H., Ho, N., & Rinaldo, A. (2024). "On least square estimation in softmax gating mixture of experts". *Proceedings of the International Conference on Machine Learning*.
- [123] OpenAI. (2023). "Gpt-4 technical report".
- [126] Paster, K., Dos Santos, M., Azerbayev, Z., & Ba, J. (2023). "Openwebmath: An open dataset of high-quality mathematical web text". arXiv preprint arXiv:2310.06786.
- [136] Roberts, N., Guo, S., Gao, Z., Namburi GNVV, S. S. S., Cromp, S., Wu, C., Duan, C., & Sala, F. (2024). "Pre-trained hybrids with mad skills". arXiv preprint arXiv:2406.00894.
- [173] Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Remez, T., Rapin, J., et al. (2023). "Code llama: Open foundation models for code". arXiv preprint arXiv:2308.12950.
- [174] Shao, Y., Jiang, Y., Kanell, T. A., Xu, P., Khattab, O., & Lam, M. S. (2024). "Assisting in writing wikipedia-like articles from scratch with large language models". arXiv preprint arXiv:2402.14204.
- [175] Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer". arXiv preprint arXiv:1701.06535.
- [124] Sukhbaatar, S., Golovneva, O., Sharma, V., Xu, H., Lin, X. V., Rozière, B., Kahn, J., Li, D., Yih, W.-T., Weston, J., & Li, X. (2024). "Branch-train-mix: Mixing expert llms into a

mixture-of-experts llm".
- [176] Together Computer. (2023). "Redpajama: An open dataset for training large language models".
- [177] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. (2023). "Llama: Open and efficient foundation language models". arXiv preprint arXiv:2302.13970.
- [178] Wan, F., Huang, X., Cai, D., Quan, X., Bi, W., & Shi, S. (2024). "Knowledge fusion of large language models". arXiv preprint arXiv:2401.10490.
- [179] Wang, X., Chen, J., Wang, Z., Zhou, Y., Zhou, Y., Lu, H., Xu, Y., He, F., Yoon, J., Lu, T., et al. (2024b). "Mementos: A comprehensive benchmark for multimodal large language model reasoning over image sequences". arXiv preprint arXiv:2401.10529.
- [60] Wang, X., Zhou, Y., Liu, X., Lu, H., Xu, Y., He, F., Yoon, J., Lu, T., Bertasius, G., Bansal, M., et al. (2024a). "Enhancing visual-language modality alignment in large vision language models via self-improvement".
- [180] Wortsman, M., Ilharco, G., Gadre, S. Y., Roelofs, R., Gontijo-Lopes, R., Morcos, A. S., Namkoong, H., Farhadi, A., Carmon, Y., Kornblith, S., et al. (2022). "Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time". *International Conference on Machine Learning*, 23965–23998.
- [181] Wu, C., Gan, Y., Ge, Y., Lu, Z., Wang, J., Feng, Y., Luo, P., & Shan, Y. (2024). "Llama pro: Progressive llama with block expansion". arXiv preprint arXiv:2401.02415.
- [182] Yadav, P., Tam, D., Choshen, L., Raffel, C. A., & Bansal, M. (2024). "Ties-merging: Resolving interference when merging models". *Advances in Neural Information Processing Systems, 36*.
- [183] Yu, L., Yu, B., Yu, H., Huang, F., & Li, Y. (2024). "Language models are super mario: Absorbing abilities from homologous models as a free lunch". *Forty-first International Conference on Machine Learning*.
- [75] Yu, L., Jiang, W., Shi, H., Yu, J., Liu, Z., Zhang, Y., Kwok, J. T., Li, Z., Weller, A., & Liu, W. (2023). "Metamath: Bootstrap your own mathematical questions for large language models". arXiv preprint arXiv:2309.12284.
- [184] Zhang, P., Zeng, G., Wang, T., & Lu, W. (2024). "Tinyllama: An open-source small language model". arXiv preprint arXiv:2401.02384.
- [185] Zhang, X., Shen, Y., Huang, Z., Zhou, J., Rong, W., & Xiong, Z. (2022). "Mixture of attention heads: Selecting attention heads per token".
- [186] Zhou, Y., & Ai, W. (2024). "Teaching-assistant-in-the-loop: Improving knowledge distillation from imperfect teacher models in low-budget scenarios". arXiv preprint arXiv:2406.05322.
- [187] Zhou, Y., Maharjan, S., & Liu, B. (2023). "Scalable prompt generation for semi-supervised learning with language models". arXiv preprint arXiv:2302.09237.
- [188] Zhou, Y., Xu, P., Liu, X., An, B., Ai, W., & Huang, F. (2024a). "Explore spurious correlations at the concept level in language models for text classification".
- [189] Zhou, Y., Zhu, J., Xu, P., Liu, X., Wang, X., Koutra, D., Ai, W., & Huang, F. (2024b). "Multi-stage balanced distillation: Addressing long-tail challenges in sequence-level knowledge distillation". arXiv preprint arXiv:2406.13114.
- [67] Zhu, J., Zhou, Y., Qian, S., He, Z., Zhao, T., & Shah, N. (2024). "Multimodal graph benchmark". arXiv preprint arXiv:2406.16321.